实验五　Spring AOP 面向切面编程

一、实验目的及任务

通过该实验，掌握 Spring 面向切面编程常用方法，掌握利用 Spring 注解方式实现面向切面编程 AOP。

二、实验环境及条件

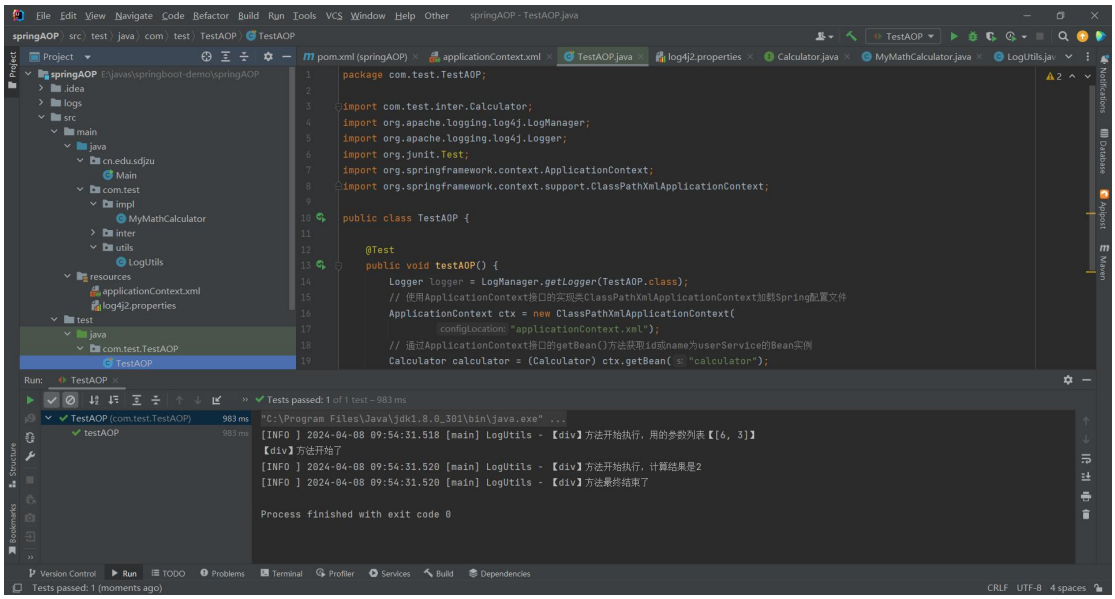
主机操作系统为 Win11，Tomcat8,jdk8 或以上版本。

三、实验实施步骤

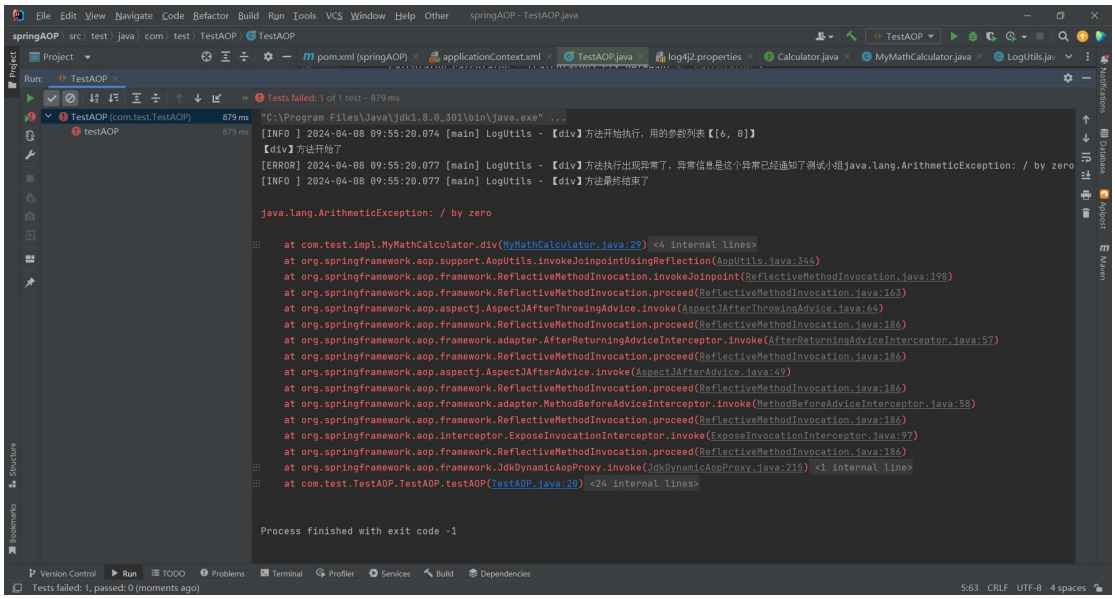
略

四、实验报告内容

1 运行测试类，修改测试数据，计算器在进行乘法和除法运算的输出结果是什么？发生除零异常时的结果是怎样的，给出截图。



报除 0 异常：



2 log4j 输出的文本中，哪些是日志切面类输出的，哪些是计算器业务类输出的？

日志切面类输出：

LogUtils - 【div】方法开始执行，用的参数列表【[6, 2]】

LogUtils - 【div】方法开始执行，计算结果是 3

LogUtils - 【div】方法最终结束了

LogUtilsPointCut - 【环绕前置通知】+div 方法开始了

LogUtilsPointCut - 【div】方法执行出现异常了，异常信息是这个异常已经通知了测试小组 java.lang.ArithmeticException: / by zero

LogUtilsPointCut - 【div】方法最终结束了

LogUtilsPointCut - 【环绕异常通知】+div 方法出现了异常 java.lang.ArithmeticException: / by zero

LogUtilsPointCut - 【环绕后置通知】+div 方法结束了

LogUtils - 【div】方法开始执行，计算结果是 null

LogUtils - 【div】方法最终结束了

LogUtilsPointCut - 【div】方法开始执行，计算结果是 2

LogUtilsPointCut - 【div】方法最终结束了

LogUtilsPointCut - 【环绕返回通知】+div 方法返回了，返回值 2

LogUtilsPointCut - 【环绕后置通知】+div 方法结束了

LogUtils - 【div】方法开始执行，计算结果是 2

LogUtils - 【div】方法最终结束了

计算器业务类输出：

【div】方法开始了

3 在本程序中，对应概念切面，切入点，前置通知（增强）和后置返回通知（增强）、异常增强、最终增强的注解有哪些？基于注解的实现中，切面类包含了哪些内容，切点表达式的作用与语法是什么？

1)

概念	注解	说明
切面	@Aspect	表示一个类是切面类，包含了增强（Advice）和切入点（Pointcut）的定义。
切入点	@Before、@After、@AfterReturning、@AfterThrowing. @Pointcut	表示在特定的连接点（Join Point）上执行的通知（Advice）。@Before 表示在方法执行之前执行，@After 表示在方法执行之后执行，@AfterReturning 表示在方法正常返回之后执行，@AfterThrowing 表示在方法抛出异常之后执行。@Pointcut 定义一个可重用的切入点表达式，用于指定哪些方法将会被拦截，从而应用通知。
前置通知（增强）	@Before	在目标方法执行之前执行的通知。
后置返回通知（增强）	@AfterReturning	在目标方法正常返回之后执行的通知。
异常增强	@AfterThrowing	在目标方法抛出异常之后执行的通知。
最终增强	@After	在目标方法执行之后执行的通知，无论是否抛出异常。
环绕通知	@Around	围绕着目标方法执行的通知，可以在目标方法执行之前和之后执行自定义逻辑。

2) 切面类包含通知方法（Advice Methods）：这些方法包含了实际的增强逻辑，如日志记录、安全检查等。以及切入点表达式（Pointcut Expressions）：这些表达式定义了哪些方法将会被拦截，从而应用通知。

3) 切点表达式的作用是定义哪些方法匹配这个表达式，从而触发相应的通知。

4) 切点表达式语法:

```
execution(modifiers-pattern? ret-type-pattern declaring-type-pattern?
name-pattern(param-pattern) throws-pattern?)
```

- modifiers-pattern：方法的访问修饰符，如 public、private 等，可选。
- ret-type-pattern：方法的返回值类型，必填。
- declaring-type-pattern：方法所在的类的全限定名，可选。

- `name-pattern`：方法的名字，必填。
- `param-pattern`：方法的参数类型，必填。可以使用 `..` 来表示任意数量的参数。
- `throws-pattern`：方法可能抛出的异常类型，可选。

4 运行测试方法时，两个切面类的运行次序是怎样的，用哪个注解来修改切面类运行次序？

运行次序：在 Spring AOP 中，切面（Aspect）的执行顺序可以通过 @Order 注解来指定。@Order 注解接受一个整数作为参数，数值越小，切面的执行顺序越靠前。LogUtils 类会先于 LogUtilsPointCut 类执行，因为 LogUtils 的顺序值更小。

修改运行次序：可以修改@order 注解，如果同一个连接点上应用了多个切面，那么这些切面的执行顺序将遵循它们各自的 @Order 注解指定的顺序。如果没有指定 @Order，则按照默认的加载顺序执行。